

Table of Contents

- [1. Executive Summary](#)
- [2. Vulnerability Overview](#)
- [3. Detailed Findings](#)
- [4. Remediation Guide](#)
- [5. Methodology](#)

1. Executive Summary

Overall Risk Score

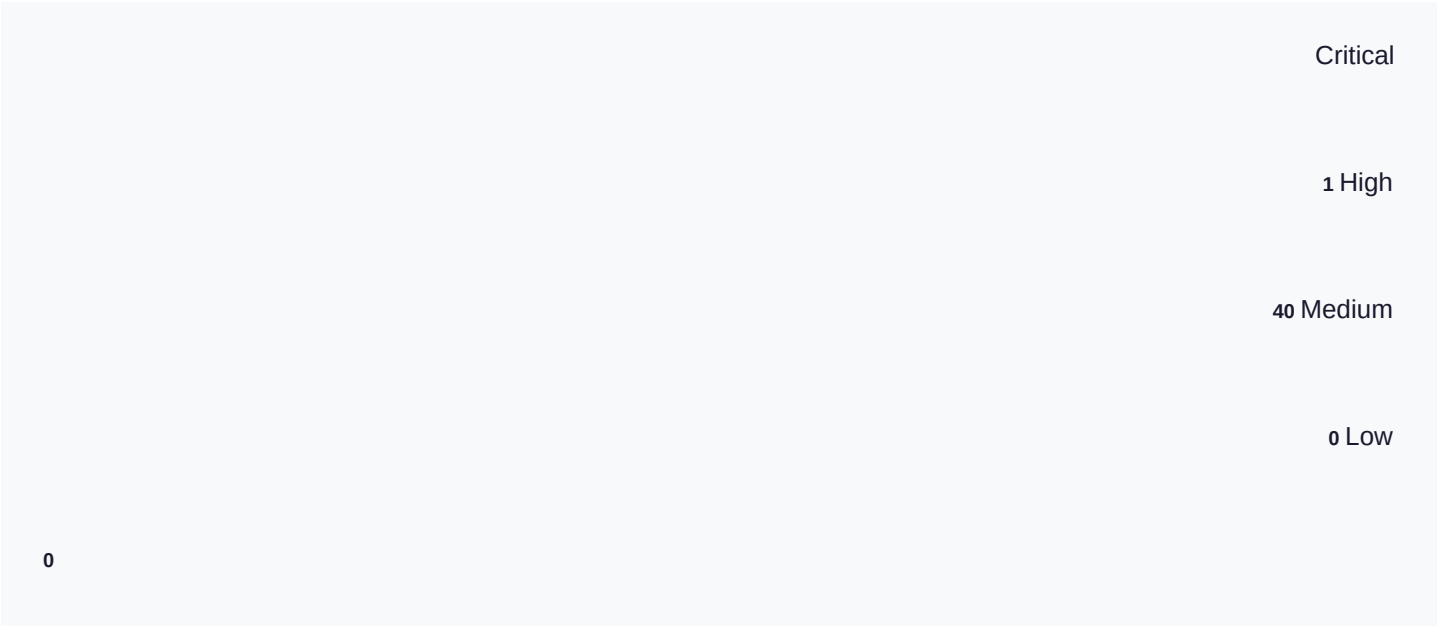
Critical risk - Severe vulnerabilities found. Urgent remediation required.

Property	Value
Target URL	https://pentest-ground.com:81

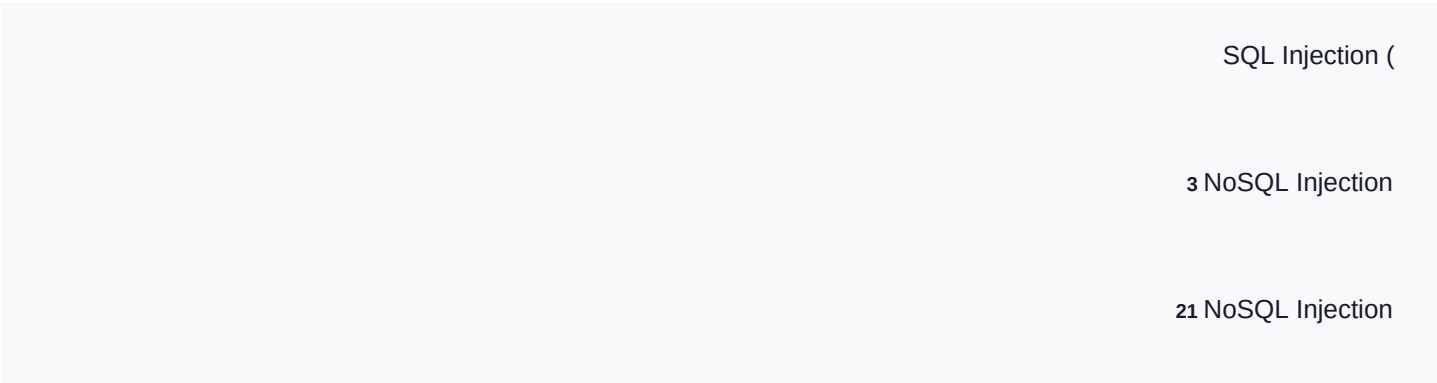
Scan Start	2026-01-27 22:08:47
Scan End	2026-01-27 22:13:25
Status	completed
Total Vulnerabilities	41

2. Vulnerability Overview

Severity Distribution



Vulnerability Types



1	7 XXE (Error-base)				
	3 XSS (event_hand				
	3 Header Injectio				
	3 SSRF (aws)				

Vulnerabilities Summary Table

#	Type	Severity	URL	Parameter	Confidence
1	SQL Injection (Boolean-based Blind)	HIGH	https://pentest-ccqueryl.com:81/search	search	90%
2	NoSQL Injection (operator)	HIGH	https://pentest-ccqueryl.com:81/search	search	85%
3	NoSQL Injection (operator)	HIGH	https://pentest-ccqueryl.com:81/search	search	85%
4	NoSQL Injection (operator)	HIGH	https://pentest-ccqueryl.com:81/search	search	85%
5	NoSQL Injection (where)	HIGH	https://pentest-ccqueryl.com:81/search	search	85%

6	XXE (Error-based)	HIGH	https://pentest-ζqueryl.com:81/search?75%
7	XXE (Error-based)	HIGH	https://pentest-ζqueryl.com:81/search?75%
8	XXE (Error-based)	HIGH	https://pentest-ζqueryl.com:81/search?75%
9	SQL Injection (Boolean-based Blind)	HIGH	https://pentest-ζtitlel.com:81/create?90%
10	SQL Injection (Boolean-based Blind)	HIGH	https://pentest-ζcontent:om:81/create?90%
11	XSS (event_handler)	HIGH	https://pentest-ζtitlel.com:81/create?70%
12	NoSQL Injection (operator)	HIGH	https://pentest-ζtitlel.com:81/create?85%
13	NoSQL Injection (operator)	HIGH	https://pentest-ζtitlel.com:81/create?85%
14	NoSQL Injection (operator)	HIGH	https://pentest-ζtitlel.com:81/create?85%

15	NoSQL Injection (where)	HIGH	https://pentest-ctitle.com:81/create?5%
16	NoSQL Injection (operator)	HIGH	https://pentest-ctcontent:om:81/create?5%
17	NoSQL Injection (operator)	HIGH	https://pentest-ctcontent:om:81/create?5%
18	NoSQL Injection (operator)	HIGH	https://pentest-ctcontent:om:81/create?5%
19	NoSQL Injection (where)	HIGH	https://pentest-ctcontent:om:81/create?5%
20	Header Injection (CRLF)	HIGH	https://pentest-ctusername:m:81/login?0%
21	Header Injection (CRLF)	HIGH	https://pentest-ctpassword:m:81/login?0%
22	Header Injection (CRLF)	HIGH	https://pentest-ctremember_me:1/login?0%
23	XSS (event_handler)	HIGH	https://pentest-cttitle.com:81/2/eq?0%

24	NoSQL Injection (operator)	HIGH	https://pentest-ctitlel.com:81/2/e85%
25	NoSQL Injection (operator)	HIGH	https://pentest-ctitlel.com:81/2/e85%
26	NoSQL Injection (operator)	HIGH	https://pentest-ctitlel.com:81/2/e85%
27	NoSQL Injection (where)	HIGH	https://pentest-ctitlel.com:81/2/e85%
28	NoSQL Injection (operator)	HIGH	https://pentest-ctcontent:om:81/2/e85%
29	NoSQL Injection (operator)	HIGH	https://pentest-ctcontent:om:81/2/e85%
30	NoSQL Injection (operator)	HIGH	https://pentest-ctcontent:om:81/2/e85%
31	NoSQL Injection (where)	HIGH	https://pentest-ctcontent:om:81/2/e85%
32	XSS (event_handler)	HIGH	https://pentest-ctitlel.com:81/1/e70%

33	NoSQL Injection (operator)	HIGH	https://pentest-ctitlel.com:81/1/e85%
34	NoSQL Injection (operator)	HIGH	https://pentest-ctitlel.com:81/1/e85%
35	NoSQL Injection (operator)	HIGH	https://pentest-ctitlel.com:81/1/e85%
36	NoSQL Injection (where)	HIGH	https://pentest-ctitlel.com:81/1/e85%
37	NoSQL Injection (operator)	HIGH	https://pentest-ctcontent:om:81/1/e85%
38	NoSQL Injection (operator)	HIGH	https://pentest-ctcontent:om:81/1/e85%
39	NoSQL Injection (operator)	HIGH	https://pentest-ctcontent:om:81/1/e85%
40	NoSQL Injection (where)	HIGH	https://pentest-ctcontent:om:81/1/e85%
41	SSRF (aws)	CRITICAL	https://pentest-creference:81/cre85%

3. Detailed Findings

1. SQL Injection (Boolean-based Blind)

HIGH

Affected URL

https://pentest-ground.com:81/search

Vulnerable Parameter

query

Payload Used

' OR '1'='1' --

Description

Vulnerability found in parameter 'query' at https://pentest-ground.com:81/search

Evidence

Payload " OR '1'='1' --" triggered vulnerability detection

Confidence Level

90%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

2. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/search

Vulnerable Parameter

query

Payload Used

{"\$gt": ""}

Description

Vulnerability found in parameter 'query' at https://pentest-ground.com:81/search

Evidence

Payload '{"\$gt": ""}' triggered vulnerability detection

Confidence Level
85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

3. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/search

Vulnerable Parameter

query

Payload Used

{"\$ne": null}

Description

Vulnerability found in parameter 'query' at https://pentest-ground.com:81/search

Evidence

Payload '{"\$ne": null}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

4. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/search

Vulnerable Parameter

query

Payload Used
<pre>{"\$ne": ""}</pre>
Description
Vulnerability found in parameter 'query' at https://pentest-ground.com:81/search
Evidence
Payload '{"\$ne": ""}' triggered vulnerability detection
Confidence Level
85%

How to Fix
• Use parameterized queries (prepared statements) for ALL database queries • Use ORM frameworks that handle parameterization automatically • Apply the principle of least privilege to database accounts • Validate and sanitize all user input • Use stored procedures with parameterized inputs • Implement Web Application Firewall (WAF) as defense-in-depth

5. NoSQL Injection (where)

HIGH

Affected URL
https://pentest-ground.com:81/search
Vulnerable Parameter
query
Payload Used
<pre>{"\$where": "1==1"}</pre>
Description
Vulnerability found in parameter 'query' at https://pentest-ground.com:81/search
Evidence
Payload '{"\$where": "1==1"}' triggered vulnerability detection
Confidence Level
85%

How to Fix
• Use parameterized queries (prepared statements) for ALL database queries • Use ORM frameworks that handle parameterization automatically • Apply the principle of least privilege to database accounts • Validate and sanitize all user input • Use stored procedures with parameterized inputs • Implement Web Application Firewall (WAF) as defense-in-depth

6. XXE (Error-based)

HIGH

Affected URL

https://pentest-ground.com:81/search

Vulnerable Parameter

query

Payload Used

]>&xxe;...

Description

Vulnerability found in parameter 'query' at https://pentest-ground.com:81/search

Evidence

Payload ']>&xxe;...' triggered vulnerability detection

Confidence Level

75%

How to Fix

- Disable DTDs (Document Type Definitions) completely
- Disable external entity processing
- Use less complex data formats (JSON) when possible
- Validate and sanitize XML input
- Keep XML processors updated

7. XXE (Error-based)

HIGH

Affected URL

https://pentest-ground.com:81/search

Vulnerable Parameter

query

Payload Used

]>&xxe;

Description

Vulnerability found in parameter 'query' at https://pentest-ground.com:81/search

Evidence

Payload ']>&xxe;'

Confidence Level

75%

How to Fix

- Disable DTDs (Document Type Definitions) completely
- Disable external entity processing
- Use less complex data formats (JSON) when possible
- Validate and sanitize XML input
- Keep XML processors updated

8. XXE (Error-based)

HIGH

Affected URL

https://pentest-ground.com:81/search

Vulnerable Parameter

query

Payload Used

]>&xxe;...

Description

Vulnerability found in parameter 'query' at https://pentest-ground.com:81/search

Evidence

Payload ']>&xxe;...' triggered vulnerability detection

Confidence Level

75%

How to Fix

- Disable DTDs (Document Type Definitions) completely
- Disable external entity processing
- Use less complex data formats (JSON) when possible
- Validate and sanitize XML input
- Keep XML processors updated

9. SQL Injection (Boolean-based Blind)

HIGH

Affected URL

https://pentest-ground.com:81/create

Vulnerable Parameter

title

Payload Used

' OR '1'='1' --

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/create
Evidence
Payload '' OR '1'='1' --' triggered vulnerability detection
Confidence Level
90%

How to Fix
• Use parameterized queries (prepared statements) for ALL database queries
• Use ORM frameworks that handle parameterization automatically
• Apply the principle of least privilege to database accounts
• Validate and sanitize all user input
• Use stored procedures with parameterized inputs
• Implement Web Application Firewall (WAF) as defense-in-depth

10. SQL Injection (Boolean-based Blind)

HIGH

Affected URL
https://pentest-ground.com:81/create
Vulnerable Parameter
content
Payload Used
' OR '1'='1' --
Description
Vulnerability found in parameter 'content' at https://pentest-ground.com:81/create
Evidence
Payload '' OR '1'='1' --' triggered vulnerability detection
Confidence Level
90%

How to Fix
• Use parameterized queries (prepared statements) for ALL database queries
• Use ORM frameworks that handle parameterization automatically
• Apply the principle of least privilege to database accounts
• Validate and sanitize all user input
• Use stored procedures with parameterized inputs
• Implement Web Application Firewall (WAF) as defense-in-depth

11. XSS (event_handler)

HIGH

Affected URL

https://pentest-ground.com:81/create

Vulnerable Parameter

title

Payload Used

">

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/create

Evidence

Payload '">' triggered vulnerability detection

Confidence Level

70%

How to Fix

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies
- Use modern frameworks that auto-escape output (React, Vue, Angular)
- Validate and sanitize all input on the server side
- Use `textContent` instead of `innerHTML` when setting element content

12. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/create

Vulnerable Parameter

title

Payload Used

{"\$gt": ""}

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/create

Evidence

Payload '{"\$gt": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically

- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

13. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/create

Vulnerable Parameter

title

Payload Used

```
{"$ne": null}
```

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/create

Evidence

Payload '{"\$ne": null}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

14. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/create

Vulnerable Parameter

title

Payload Used

```
{"$ne": ""}
```

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/create

Evidence

Payload '{"\$ne": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

15. NoSQL Injection (where)

HIGH

Affected URL

https://pentest-ground.com:81/create

Vulnerable Parameter

title

Payload Used

'{"\$where": "1==1"}'

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/create

Evidence

Payload '{"\$where": "1==1"}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

16. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/create

Vulnerable Parameter

content
Payload Used
<pre>{"\$gt": ""}</pre>
Description
Vulnerability found in parameter 'content' at https://pentest-ground.com:81/create
Evidence
Payload '{"\$gt": ""}' triggered vulnerability detection
Confidence Level
85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

17. NoSQL Injection (operator)

HIGH

Affected URL
https://pentest-ground.com:81/create
Vulnerable Parameter
content
Payload Used
<pre>{"\$ne": null}</pre>
Description
Vulnerability found in parameter 'content' at https://pentest-ground.com:81/create
Evidence
Payload '{"\$ne": null}' triggered vulnerability detection
Confidence Level
85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs

- Implement Web Application Firewall (WAF) as defense-in-depth

18. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/create

Vulnerable Parameter

content

Payload Used

```
{"$ne": ""}
```

Description

Vulnerability found in parameter 'content' at https://pentest-ground.com:81/create

Evidence

Payload '{"\$ne": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

19. NoSQL Injection (where)

HIGH

Affected URL

https://pentest-ground.com:81/create

Vulnerable Parameter

content

Payload Used

```
{"$where": "1==1"}
```

Description

Vulnerability found in parameter 'content' at https://pentest-ground.com:81/create

Evidence

Payload '{"\$where": "1==1"}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

20. Header Injection (CRLF)

HIGH

Affected URL

https://pentest-ground.com:81/login

Vulnerable Parameter

username

Payload Used

test Set-Cookie: injected=true

Description

Vulnerability found in parameter 'username' at https://pentest-ground.com:81/login

Evidence

Payload 'test Set-Cookie: injected=true' triggered vulnerability detection

Confidence Level

90%

How to Fix

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters
- Use framework functions that sanitize header values
- Validate input against strict patterns

21. Header Injection (CRLF)

HIGH

Affected URL

https://pentest-ground.com:81/login

Vulnerable Parameter

password

Payload Used

test Set-Cookie: injected=true

Description
Vulnerability found in parameter 'password' at https://pentest-ground.com:81/login
Evidence
Payload 'test Set-Cookie: injected=true' triggered vulnerability detection
Confidence Level
90%

How to Fix
• Never include raw user input in HTTP headers
• Strip or encode CR (\r) and LF (\n) characters
• Use framework functions that sanitize header values
• Validate input against strict patterns

22. Header Injection (CRLF)

HIGH

Affected URL
https://pentest-ground.com:81/login
Vulnerable Parameter
remember_me
Payload Used
test Set-Cookie: injected=true
Description
Vulnerability found in parameter 'remember_me' at https://pentest-ground.com:81/login
Evidence
Payload 'test Set-Cookie: injected=true' triggered vulnerability detection
Confidence Level
90%

How to Fix
• Never include raw user input in HTTP headers
• Strip or encode CR (\r) and LF (\n) characters
• Use framework functions that sanitize header values
• Validate input against strict patterns

23. XSS (event_handler)

HIGH

Affected URL
https://pentest-ground.com:81/2/edit
Vulnerable Parameter

title
Payload Used
">
Description
Vulnerability found in parameter 'title' at https://pentest-ground.com:81/2/edit
Evidence
Payload '">' triggered vulnerability detection
Confidence Level
70%

How to Fix

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies
- Use modern frameworks that auto-escape output (React, Vue, Angular)
- Validate and sanitize all input on the server side
- Use `textContent` instead of `innerHTML` when setting element content

24. NoSQL Injection (operator)

HIGH

Affected URL
https://pentest-ground.com:81/2/edit
Vulnerable Parameter
title
Payload Used
{"\$gt": ""}
Description
Vulnerability found in parameter 'title' at https://pentest-ground.com:81/2/edit
Evidence
Payload '{"\$gt": ""}' triggered vulnerability detection
Confidence Level
85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs

- Implement Web Application Firewall (WAF) as defense-in-depth

25. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/2/edit

Vulnerable Parameter

title

Payload Used

```
{"$ne": null}
```

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/2/edit

Evidence

Payload '{"\$ne": null}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

26. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/2/edit

Vulnerable Parameter

title

Payload Used

```
{"$ne": ""}
```

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/2/edit

Evidence

Payload '{"\$ne": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

27. NoSQL Injection (where)

HIGH

Affected URL

https://pentest-ground.com:81/2/edit

Vulnerable Parameter

title

Payload Used

{"\$where": "1==1"}

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/2/edit

Evidence

Payload '{"\$where": "1==1"}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

28. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/2/edit

Vulnerable Parameter

content

Payload Used

```
{"$gt": ""}
```

Description

Vulnerability found in parameter 'content' at <https://pentest-ground.com:81/2/edit>

Evidence

Payload '{"\$gt": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

29. NoSQL Injection (operator)

HIGH

Affected URL

<https://pentest-ground.com:81/2/edit>

Vulnerable Parameter

content

Payload Used

```
{"$ne": null}
```

Description

Vulnerability found in parameter 'content' at <https://pentest-ground.com:81/2/edit>

Evidence

Payload '{"\$ne": null}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

30. NoSQL Injection (operator)

HIGH

Affected URL
https://pentest-ground.com:81/2/edit
Vulnerable Parameter
content
Payload Used
{"\$ne": ""}
Description
Vulnerability found in parameter 'content' at https://pentest-ground.com:81/2/edit
Evidence
Payload '{"\$ne": ""}' triggered vulnerability detection
Confidence Level
85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

31. NoSQL Injection (where)

HIGH

Affected URL
https://pentest-ground.com:81/2/edit
Vulnerable Parameter
content
Payload Used
{"\$where": "1==1"}
Description
Vulnerability found in parameter 'content' at https://pentest-ground.com:81/2/edit
Evidence
Payload '{"\$where": "1==1"}' triggered vulnerability detection
Confidence Level
85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries

- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

32. XSS (event_handler)

HIGH

Affected URL

https://pentest-ground.com:81/1/edit

Vulnerable Parameter

title

Payload Used

">

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/1/edit

Evidence

Payload '">' triggered vulnerability detection

Confidence Level

70%

How to Fix

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies
- Use modern frameworks that auto-escape output (React, Vue, Angular)
- Validate and sanitize all input on the server side
- Use `textContent` instead of `innerHTML` when setting element content

33. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/1/edit

Vulnerable Parameter

title

Payload Used

{"\$gt": ""}

Description

Vulnerability found in parameter 'title' at https://pentest-ground.com:81/1/edit

Evidence
Payload '{"\$gt": ""}' triggered vulnerability detection
Confidence Level
85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

34. NoSQL Injection (operator)

HIGH

Affected URL
https://pentest-ground.com:81/1/edit
Vulnerable Parameter
title
Payload Used
{"\$ne": null}
Description
Vulnerability found in parameter 'title' at https://pentest-ground.com:81/1/edit
Evidence
Payload '{"\$ne": null}' triggered vulnerability detection
Confidence Level
85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

35. NoSQL Injection (operator)

HIGH

Affected URL
https://pentest-ground.com:81/1/edit

Vulnerable Parameter
title
Payload Used
{"\$ne": ""}
Description
Vulnerability found in parameter 'title' at https://pentest-ground.com:81/1/edit
Evidence
Payload '{"\$ne": ""}' triggered vulnerability detection
Confidence Level
85%

How to Fix
• Use parameterized queries (prepared statements) for ALL database queries
• Use ORM frameworks that handle parameterization automatically
• Apply the principle of least privilege to database accounts
• Validate and sanitize all user input
• Use stored procedures with parameterized inputs
• Implement Web Application Firewall (WAF) as defense-in-depth

36. NoSQL Injection (where)

HIGH

Affected URL
https://pentest-ground.com:81/1/edit
Vulnerable Parameter
title
Payload Used
{"\$where": "1==1"}
Description
Vulnerability found in parameter 'title' at https://pentest-ground.com:81/1/edit
Evidence
Payload '{"\$where": "1==1"}' triggered vulnerability detection
Confidence Level
85%

How to Fix
• Use parameterized queries (prepared statements) for ALL database queries
• Use ORM frameworks that handle parameterization automatically
• Apply the principle of least privilege to database accounts
• Validate and sanitize all user input

- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

37. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/1/edit

Vulnerable Parameter

content

Payload Used

```
{"$gt": ""}
```

Description

Vulnerability found in parameter 'content' at https://pentest-ground.com:81/1/edit

Evidence

Payload '{"\$gt": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

38. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/1/edit

Vulnerable Parameter

content

Payload Used

```
{"$ne": null}
```

Description

Vulnerability found in parameter 'content' at https://pentest-ground.com:81/1/edit

Evidence

Payload '{"\$ne": null}' triggered vulnerability detection

Confidence Level
85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

39. NoSQL Injection (operator)

HIGH

Affected URL

https://pentest-ground.com:81/1/edit

Vulnerable Parameter

content

Payload Used

{"\$ne": ""}

Description

Vulnerability found in parameter 'content' at https://pentest-ground.com:81/1/edit

Evidence

Payload '{"\$ne": ""}' triggered vulnerability detection

Confidence Level

85%

How to Fix

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

40. NoSQL Injection (where)

HIGH

Affected URL

https://pentest-ground.com:81/1/edit

Vulnerable Parameter

content

Payload Used
<pre>{"\$where": "1==1"}</pre>
Description
Vulnerability found in parameter 'content' at https://pentest-ground.com:81/1/edit
Evidence
Payload '{"\$where": "1==1"}' triggered vulnerability detection
Confidence Level
85%

How to Fix
• Use parameterized queries (prepared statements) for ALL database queries • Use ORM frameworks that handle parameterization automatically • Apply the principle of least privilege to database accounts • Validate and sanitize all user input • Use stored procedures with parameterized inputs • Implement Web Application Firewall (WAF) as defense-in-depth

41. SSRF (aws)

CRITICAL

Affected URL
https://pentest-ground.com:81/create
Vulnerable Parameter
reference
Payload Used
http://169.254.169.254/latest/meta-data/
Description
Vulnerability found in parameter 'reference' at https://pentest-ground.com:81/create
Evidence
Payload 'http://169.254.169.254/latest/meta-data/' triggered vulnerability detection
Confidence Level
85%

How to Fix
• Validate and sanitize all user-supplied URLs • Use allowlist of permitted domains/IPs • Block requests to private IP ranges and localhost • Disable unnecessary URL schemes (file://, gopher://) • Use network segmentation • Block access to cloud metadata endpoints

4. Remediation Guide

This section provides detailed guidance on how to fix the vulnerabilities found during the scan. Each vulnerability type includes an explanation of how the attack works and code examples for remediation.

SQL Injection (Boolean-based Blind)

3 Finding(s)

What is SQL Injection (Boolean-based Blind)?

SQL Injection occurs when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without authorization.

How the Attack Works

1. Application builds SQL queries using string concatenation with user input 2. Attacker provides malicious input containing SQL syntax 3. The database executes the modified query 4. Attacker can read, modify, or delete data; bypass authentication

Potential Impact

- Complete database compromise
- Data theft (credentials, personal info, financial data)
- Data modification or deletion
- Authentication bypass
- Remote code execution (in some cases)
- Full server compromise

Remediation Steps

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python) query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query) # VULNERABLE CODE (PHP) $query = "SELECT * FROM users WHERE id = " .
$_GET['id']; $result = mysqli_query($conn, $query);
```

SECURE CODE:

```
# SECURE CODE (Python with parameterized query) query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,)) # SECURE CODE (Python with SQLAlchemy ORM) user =
session.query(User).filter(User.username == username).first() # SECURE CODE (PHP with PDO) $stmt =
$pdo->prepare("SELECT * FROM users WHERE id = ?"); $stmt->execute($_GET['id']);
```

References

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

CWE: CWE-89 OWASP: A03:2021 - Injection

NoSQL Injection (operator)

21 Finding(s)

What is NoSQL Injection (operator)?

SQL Injection occurs when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without authorization.

How the Attack Works

1. Application builds SQL queries using string concatenation with user input 2. Attacker provides malicious input containing SQL syntax 3. The database executes the modified query 4. Attacker can read, modify, or delete data; bypass authentication

Potential Impact

- Complete database compromise
- Data theft (credentials, personal info, financial data)
- Data modification or deletion
- Authentication bypass
- Remote code execution (in some cases)
- Full server compromise

Remediation Steps

- Use parameterized queries (prepared statements) for ALL database queries
- Use ORM frameworks that handle parameterization automatically
- Apply the principle of least privilege to database accounts
- Validate and sanitize all user input
- Use stored procedures with parameterized inputs
- Implement Web Application Firewall (WAF) as defense-in-depth

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python) query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query) # VULNERABLE CODE (PHP) $query = "SELECT * FROM users WHERE id = " .
$_GET['id']; $result = mysqli_query($conn, $query);
```

SECURE CODE:

```
# SECURE CODE (Python with parameterized query) query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,)) # SECURE CODE (Python with SQLAlchemy ORM) user =
session.query(User).filter(User.username == username).first() # SECURE CODE (PHP with PDO) $stmt =
$pdo->prepare("SELECT * FROM users WHERE id = ?"); $stmt->execute($_GET['id']);
```

References

- https://owasp.org/www-community/attacks/SQL_Injection
- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

CWE: CWE-89 OWASP: A03:2021 - Injection

NoSQL Injection (where)

7 Finding(s)

What is NoSQL Injection (where)?

SQL Injection occurs when untrusted data is sent to an interpreter as part of a command or query. The attacker's hostile data can trick the interpreter into executing unintended commands or accessing data without authorization.

How the Attack Works

1. Application builds SQL queries using string concatenation with user input 2. Attacker provides malicious input containing SQL syntax 3. The database executes the modified query 4. Attacker can read, modify, or delete data; bypass authentication

Potential Impact
• Complete database compromise • Data theft (credentials, personal info, financial data) • Data modification or deletion • Authentication bypass • Remote code execution (in some cases) • Full server compromise

Remediation Steps
• Use parameterized queries (prepared statements) for ALL database queries • Use ORM frameworks that handle parameterization automatically • Apply the principle of least privilege to database accounts • Validate and sanitize all user input • Use stored procedures with parameterized inputs • Implement Web Application Firewall (WAF) as defense-in-depth

Code Examples
VULNERABLE CODE: # VULNERABLE CODE (Python) query = "SELECT * FROM users WHERE username = '" + username + "'" cursor.execute(query) # VULNERABLE CODE (PHP) \$query = "SELECT * FROM users WHERE id = " . \$_GET['id']; \$result = mysqli_query(\$conn, \$query); SECURE CODE: # SECURE CODE (Python with parameterized query) query = "SELECT * FROM users WHERE username = ?" cursor.execute(query, (username,)) # SECURE CODE (Python with SQLAlchemy ORM) user = session.query(User).filter(User.username == username).first() # SECURE CODE (PHP with PDO) \$stmt = \$pdo->prepare("SELECT * FROM users WHERE id = ?"); \$stmt->execute(\$_GET['id']);
References
• https://owasp.org/www-community/attacks/SQL_Injection • https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

CWE: CWE-89 OWASP: A03:2021 - Injection
XXE (Error-based)
3 Finding(s)

What is XXE (Error-based)?
XXE occurs when XML input containing a reference to an external entity is processed by a weakly configured XML parser. Attackers can use external entities to access files, perform SSRF, or cause denial of service.
How the Attack Works
1. Application parses XML from user input 2. Attacker includes DOCTYPE with external entity definition 3. Parser resolves the external entity (file, URL) 4. Content of file/URL is included in response or error

Potential Impact
• Local file disclosure • Server-side request forgery • Denial of service (billion laughs attack) • Port scanning • Remote code execution (rare)

Remediation Steps
• Disable DTDs (Document Type Definitions) completely

- Disable external entity processing
- Use less complex data formats (JSON) when possible
- Validate and sanitize XML input
- Keep XML processors updated

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python) from lxml import etree xml_data = request.data doc = etree.fromstring(xml_data) # Vulnerable to XXE
```

SECURE CODE:

```
# SECURE CODE (Python with lxml) from lxml import etree xml_data = request.data parser = etree.XMLParser( resolve_entities=False, # Disable entity resolution no_network=True, # Disable network access dtd_validation=False, # Disable DTD validation load_dtd=False # Don't load DTD ) doc = etree.fromstring(xml_data, parser=parser) # SECURE CODE (Python with defusedxml - recommended) import defusedxml.ElementTree as ET doc = ET.fromstring(xml_data) # Safe by default
```

References

- [https://owasp.org/www-community/vulnerabilities/XML_External_Entity_\(XXE\)_Processing](https://owasp.org/www-community/vulnerabilities/XML_External_Entity_(XXE)_Processing)

CWE: CWE-611 **OWASP:** A05:2021 - Security Misconfiguration

XSS (event_handler)

3 Finding(s)

What is XSS (event_handler)?

Cross-Site Scripting (XSS) occurs when an application includes untrusted data in a web page without proper validation or escaping. This allows attackers to execute malicious scripts in victims' browsers.

How the Attack Works

1. Attacker identifies an input field that reflects user data in the page 2. Attacker crafts a malicious payload containing JavaScript code 3. When a victim views the page, the script executes in their browser 4. The script can steal cookies, session tokens, or perform actions as the user

Potential Impact

- Session hijacking and account takeover
- Theft of sensitive data (cookies, tokens)
- Defacement of web pages
- Phishing attacks using trusted domain
- Malware distribution

Remediation Steps

- Encode all user input before displaying in HTML (use HTML entity encoding)
- Use Content-Security-Policy (CSP) headers to restrict script sources
- Implement HttpOnly and Secure flags on session cookies
- Use modern frameworks that auto-escape output (React, Vue, Angular)
- Validate and sanitize all input on the server side
- Use `textContent` instead of `innerHTML` when setting element content

Code Examples

VULNERABLE CODE:

SECURE CODE:

```
// Server-side (Python/Flask example): from markupsafe import escape @app.route('/display') def display(): user_input = request.args.get('input', '') safe_input = escape(user_input) return
```

```
render_template('page.html', content=safe_input)
```

References

- <https://owasp.org/www-community/attacks/xss/>
- https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

CWE: CWE-79 OWASP: A03:2021 - Injection

Header Injection (CRLF)

3 Finding(s)

What is Header Injection (CRLF)?

Header Injection occurs when user input is included in HTTP response headers without proper sanitization. By injecting CRLF (\r\n) sequences, attackers can add arbitrary headers or split the response.

How the Attack Works

1. Application includes user input in response headers 2. Attacker provides input with CRLF characters 3. New headers are injected into the response 4. Can lead to XSS, cache poisoning, or session fixation

Potential Impact

- Cross-site scripting via injected content
- Cache poisoning
- Session fixation attacks
- HTTP response splitting

Remediation Steps

- Never include raw user input in HTTP headers
- Strip or encode CR (\r) and LF (\n) characters
- Use framework functions that sanitize header values
- Validate input against strict patterns

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python/Flask) @app.route('/redirect') def redirect_user(): location = request.args.get('url') response = make_response() response.headers['Location'] = location return response # Attacker: ?url=http://example.com%0d%0aSet-Cookie:%20session=evil
```

SECURE CODE:

```
# SECURE CODE (Python/Flask) import re @app.route('/redirect') def redirect_user(): location = request.args.get('url', '/') # Remove any CRLF characters safe_location = re.sub(r'[\r\n]', '', location) # Or validate URL format strictly if not re.match(r'^https?://[a-zA-Z0-9.-]+/', safe_location): safe_location = '/' return redirect(safe_location)
```

References

- https://owasp.org/www-community/attacks/HTTP_Response_Splitting

CWE: CWE-113 OWASP: A03:2021 - Injection

SSRF (aws)

1 Finding(s)

What is SSRF (aws)?

SSRF occurs when an application fetches a remote resource using a user-supplied URL without proper validation. Attackers can make requests to internal services, cloud metadata endpoints, or other protected resources.

How the Attack Works

1. Application accepts URL from user and fetches it server-side 2. Attacker provides URL to internal service (localhost, 169.254.169.254) 3. Server fetches the internal resource 4. Attacker receives sensitive internal data or performs actions

Potential Impact

- Access to internal services
- Cloud metadata credential theft (AWS keys, etc.)
- Internal network scanning
- Bypass of firewalls and access controls
- Remote code execution via internal services

Remediation Steps

- Validate and sanitize all user-supplied URLs
- Use allowlist of permitted domains/IPs
- Block requests to private IP ranges and localhost
- Disable unnecessary URL schemes (file://, gopher://)
- Use network segmentation
- Block access to cloud metadata endpoints

Code Examples

VULNERABLE CODE:

```
# VULNERABLE CODE (Python)
import requests
url = request.args.get('url')
response = requests.get(url)
return response.text

# Attacker:
?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

SECURE CODE:

```
# SECURE CODE (Python)
import requests
import ipaddress
from urllib.parse import urlparse

BLOCKED_HOSTS = ['localhost', '127.0.0.1', '0.0.0.0']
ALLOWED_SCHEMES = ['http', 'https']

def is_safe_url(url):
    parsed = urlparse(url)
    # Check scheme
    if parsed.scheme not in ALLOWED_SCHEMES:
        return False
    # Check for blocked hosts
    if parsed.hostname in BLOCKED_HOSTS:
        return False
    # Check for private IP ranges
    try:
        ip = ipaddress.ip_address(parsed.hostname)
        if ip.is_private or ip.is_loopback or ip.is_link_local:
            return False
    except ValueError:
        pass
    # Not an IP address
    return True

url = request.args.get('url')
if not is_safe_url(url):
    abort(400, 'Invalid URL')
response = requests.get(url, timeout=5)
```

References

- https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/

CWE: CWE-918 OWASP: A10:2021 - Server-Side Request Forgery

5. Methodology

This automated scan was performed using the Web Vulnerability Scanner, which employs the following techniques to discover and test for vulnerabilities:

Scanning Phases

Phase	Description
-------	-------------

1. Discovery	Crawls the target website to discover pages, forms, API endpoints, and parameters using browser automation (Playwright).
2. Analysis	Analyzes discovered elements to identify potential injection points and build a map of testable targets.
3. Testing	Tests each target with specialized payloads for various vulnerability types (XSS, SQLi, etc.).
4. Validation	Confirms findings using multi-stage validation to reduce false positives. Assigns confidence scores.

Vulnerability Types Tested

Cross-Site Scripting (XSS) SQL Injection NoSQL Injection Path Traversal Open Redirect SSTI Command Injection SSRF XXE Header Injection

Disclaimer: This automated scan may not detect all vulnerabilities. Manual penetration testing by qualified security professionals is recommended for comprehensive security assessment. Always obtain proper authorization before scanning.